UT_11.2_Gestión de datos

Amazon Redshift es un servicio de almacenamiento de datos optimizado para aplicaciones **de procesamiento analítico en línea (OLAP)**. Puede comenzar con solo unos pocos cientos de **gigabytes (GB)** de datos y escalar hasta un **petabyte (PB)** o más, manteniendo un alto rendimiento y rentabilidad. Su arquitectura de almacenamiento en columnas está diseñada específicamente para cargas de trabajo analíticas a gran escala, almacenando los datos por columnas en lugar de por filas. Este enfoque es especialmente eficiente para las operaciones OLAP, ya que permite a Redshift leer solo las columnas específicas necesarias para el análisis, lo que mejora significativamente el rendimiento de las consultas para operaciones como agregaciones, análisis de tendencias y comparaciones de datos históricos. El diseño en columnas también permite una mejor compresión de los datos, ya que los tipos de datos similares se almacenan juntos, lo que reduce los costes de almacenamiento y mejora el rendimiento de entrada/salida (E/S). Al diseñar su base de datos para aprovechar esta arquitectura en columnas, puede maximizar la eficiencia de sus cargas de trabajo de procesamiento analítico y lograr un rendimiento óptimo para sus operaciones de almacenamiento de datos.

Amazon Redshift es ideal para el modelado de datos dimensionales, en particular esquemas en estrella, esquemas en copo de nieve, tablas desnormalizadas, etc., que están optimizados para consultas analíticas y generación de informes. Un esquema analítico constituye la base de su modelo de datos. En este tema se explica cómo puede configurar este esquema, lo que le permitirá realizar consultas óptimas utilizando **el lenguaje de consulta estructurado (SQL)** estándar y administrar fácilmente los controles de acceso.

En este tema se tratan los siguientes procedimientos:

1. Gestión de una base de datos en Amazon Redshift
2. Gestión de un esquema en una base de datos
3. Gestión de tablas en una base de datos
4. Gestión de vistas en una base de datos
5. Gestión de vistas materializadas en una base de datos
6. Gestión de procedimientos almacenados en una base de datos

Requisitos técnicos

Para completar todas los procedimientos de este tema, necesitará un almacén de datos de Amazon Redshift (ya sea sin servidor o un clúster aprovisionado) implementado y un cliente SQL de su elección para acceder a Amazon Redshift (por ejemplo, Amazon Redshift Query Editor V2, DBeaver, SQL Workbench/J, DataGrip, etc.).

1. Gestión de una base de datos en Amazon Redshift

Amazon Redshift consta de al menos una base de datos y es el nivel más alto en la jerarquía del espacio de nombres para los objetos de la base de datos. Este procedimiento le guiará a través de los pasos necesarios para crear y gestionar una base de datos en un almacén de datos de Amazon Redshift.

Preparación

Para completar este procedimiento, necesitará todo lo mencionado en la sección *Requisitos técnicos* al principio del tema.

Cómo hacerlo...

Ahora vamos a instalar y configurar una base de datos en Amazon Redshift. Utilice el cliente SQL para conectarse a ella y ejecute los siguientes comandos:

Crearemos una nueva base de datos llamada **qa** en el almacén de datos de Amazon Redshift que tiene como propietario `awsuser` y acepta un máximo de 50 conexiones. Para ello, utilice el siguiente código:

```
CREATE DATABASE qa;
ALTER DATABASE qa OWNER TO usuario;
ALTER DATABASE qa CONNECTION LIMIT 50;
```

`OWNER` y `CONNECTION LIMIT` son parámetros opcionales. Puede encontrar la lista completa de opciones disponibles para la creación de bases de datos en este enlace: https://docs.aws.amazon.com/redshift/latest/dg/r_CREATE_DATABASE.html.

Para ver los detalles de la base de datos **qa**, puede consultar `PG_DATABASE_INFO`, como se muestra en el siguiente fragmento de código:

```
SELECT datname, datdba, datconnlimit
FROM pg_database_info
WHERE datname= 'qa';
```

Este es el resultado esperado. Contiene el nombre de la base de datos, el ID de usuario del administrador de la base de datos y el límite de conexiones para la base de datos:

```
datname datdba  datconnlimit
qa    100    50
```

Esta consulta mostrará las bases de datos que existen en Redshift. Si se crea correctamente una base de datos, aparecerá en el resultado de la consulta.

Para realizar cambios en la base de datos, como el nombre, el propietario o el límite de conexiones, utilice el siguiente comando, sustituyendo `<qauser>` por el nombre de usuario correspondiente de Amazon Redshift:

```
/* Cambiar el nombre de la base de datos */
ALTER DATABASE qa RENAME TO prod;
/* Cambiar el propietario de la base de datos */
ALTER DATABASE qa owner to <qauser>;
/* Cambiar el límite de conexiones de la base de datos */
ALTER DATABASE qa CONNECTION LIMIT 100;
```

Para verificar que los cambios se han completado correctamente, puede consultar `PG_DATABASE_INFO`, como se muestra en el siguiente fragmento de código:

```
SELECT datname, datdba, datconnlimit
FROM pg_database_info
WHERE datname =  'qa';
```

Este es el resultado esperado:

```
datname datdba datconnlimit
prod    100    100
```

Puede conectarse a la base de datos `prod` utilizando el punto final de conexión, de la siguiente manera:`<RedshiftHostname>:<Port>/prod`

Aquí, `prod` hace referencia a la base de datos a la que desea conectarse.

Desde **psql** puedes cambiar de base de datos mediante el comando **\c basededatos usuario**:

```
\c qa admin
```

Para eliminar la base de datos primero debemos verificar si tiene alguna conexión abierta con el comando:

```
SELECT procpid, usename, datname
FROM pg_stat_activity
WHERE datname = 'prod';
```

Si así fuera deberíamos primero cerrar todas la conexiones:

```
SELECT pg_terminate_backend(procpid)
FROM pg_stat_activity
WHERE datname = ‘prod'
AND procpid <> pg_backend_pid();
```

Para finalmente borrar la base de datos:

```
DROP DATABASE prod;
```

2. Gestión de un esquema en una base de datos

En Amazon Redshift, un esquema es un espacio de nombres que agrupa objetos de la base de datos, como **tablas**, **vistas**, **procedimientos almacenados**, etc. Organizar los objetos de la base de datos en un esquema es bueno para la supervisión de la seguridad y también significa que los objetos se agrupan de forma lógica. En este procedimiento, crearemos un esquema de ejemplo que se utilizará para almacenar todos los objetos de la base de datos.

Preparación

Para completar este procedimiento, necesitará lo siguiente:

- Acceso a cualquier interfaz SQL, como Query Editor V2 o un cliente SQL local
- Un punto de conexión de Amazon Redshift Data Warehouse (clúster sin servidor o aprovisionado).
- Inicie sesión como superusuario en un clúster de Amazon Redshift

Cómo hacerlo…

Los usuarios pueden crear un esquema utilizando el comando `CREATE SCHEMA` (**https://docs.aws.amazon.com/redshift/latest/dg/r_CREATE_SCHEMA.html**). Los siguientes pasos le permitirán configurar un esquema con el nombre «`finance`» y añadir el acceso necesario a los grupos:

Los grupos de usuarios son una colección de usuarios. Normalmente se utilizan para agrupar a los usuarios que necesitan los mismos permisos sobre los objetos de la base de datos. Es habitual ver roles de la organización como desarrolladores, analistas y científicos de datos como grupos. Otro patrón de agrupación habitual es un grupo por organización, como finance, auditoría, etc.

Cree los grupos `finance_grp`, `audit_grp` y `finance_admin_user` utilizando las siguientes sentencias SQL:

```
create group finance_grp;
create group audit_grp;
create user finance_admin_usr with password '<PasswordOfYourChoice>';
```

Cree un esquema denominado `finance` con una cuota de espacio de `1 TB`, con un propietario del esquema `finance_admin_usr`:

```
CREATE schema finance authorization finance_admin_usr QUOTA 1 TB;
```

También puede modificar o eliminar esquemas existentes utilizando las sentencias `ALTER SCHEMA` (**https://docs.aws.amazon.com/redshift/latest/dg/r_ALTER_SCHEMA.html**) y `DROP SCHEMA` (**https://docs.aws.amazon.com/redshift/latest/dg/r_DROP_SCHEMA.html**), respectivamente.

Para el esquema `finance`, conceda privilegios de acceso de `USAGE` y `ALL` al grupo `finance_grp`. Además, conceda acceso de lectura a las tablas del esquema utilizando un privilegio

```
GRANT USAGE on SCHEMA finance TO GROUP finance_grp;
GRANT USAGE on SCHEMA finance TO GROUP audit_grp;
GRANT ALL ON schema finance to GROUP finance_grp;
GRANT SELECT ON ALL TABLES IN SCHEMA finance TO GROUP audit_grp;
```

`USAGE` concede permiso sobre un esquema específico, lo que hace que los objetos de ese esquema sean accesibles para los usuarios. Las acciones específicas sobre estos objetos deben concederse por separado (por ejemplo, permiso `SELECT` o `UPDATE` sobre las tablas). `ALL` concede todos los privilegios posibles sobre el esquema (es decir, `CREATE`, `USAGE`, `ALTER` o `DROP`).

Puede verificar que se han creado el esquema y el grupo propietario utilizando el siguiente código:

```
select nspname as schema, usename as owner
from pg_namespace, pg_user
where pg_namespace.nspowner = pg_user.usesysid
and pg_namespace.nspname ='finance';
```

Cree una tabla `foo` dentro del esquema anteponiendo el nombre del esquema junto con la tabla, como se muestra en el siguiente comando:

```
CREATE TABLE finance.foo (bar int);
```

Ahora, para seleccionar la tabla `foo` del esquema `finance`, tendrá que anteponer el nombre del esquema junto con la tabla, como se muestra en el siguiente comando:

```
select * from finance.foo;
```

El código SQL anterior no devolverá ninguna fila, ya que no hemos cargado ningún dato en `foo`.

Asigne una ruta de búsqueda para hacer referencia cómodamente a los objetos de la base de datos directamente utilizando la notación de una sola parte, como `select * from foo;`, en lugar de requerir el uso de la notación de dos partes, como `select * from finance.foo;`. El siguiente comando establece la ruta de búsqueda en `'$user', finance, public`. Al buscar objetos no especificados con un nombre de esquema, Amazon Redshift sigue la ruta de búsqueda. Busca en los esquemas en el orden indicado en la ruta de búsqueda hasta que encuentra el objeto.

```
set search_path to '$user', finance, public;
```

Puede configurar la ruta de búsqueda utilizando el comando `set search_path` en el nivel de sesión actual o en el nivel de usuario.

Ahora, al ejecutar la siguiente consulta `SELECT` sin el calificador de esquema, se localiza automáticamente la tabla `foo` en el esquema `finance`:

```
select * from foo;
```

El código SQL anterior no devolverá ninguna fila.

Ahora, el nuevo esquema `finance` está listo para su uso y puede seguir creando nuevos objetos de base de datos en este esquema.

Nota importante

De forma predeterminada, se crea automáticamente una base de datos con un esquema `PUBLIC`. Se pueden utilizar nombres de objetos de base de datos idénticos en diferentes esquemas de la base de datos. Por ejemplo, `finance.customer` y `marketing.customer` son definiciones de tabla válidas que se pueden crear sin ningún conflicto, donde `finance` y `marketing` son nombres de esquema y `customer` es el nombre de la tabla. Los esquemas tienen la función principal de facilitar la gestión a través de esta agrupación lógica; por ejemplo, se puede conceder acceso `SELECT` a todos los objetos a nivel de esquema en lugar de a tablas individuales.

3. Gestión de tablas en una base de datos

En Amazon Redshift, puede crear una colección de tablas dentro de un esquema con entidades y atributos relacionados. Partiendo de sus requisitos empresariales, puede utilizar diferentes técnicas de modelado para crear tablas en Amazon Redshift. Puede elegir un esquema **en estrella** o **en copo de nieve** utilizando técnicas de modelado de datos **normalizadas**, **desnormalizadas** o **de almacén de datos**.

INFORMACIÓN

Más información en : https://www.datacamp.com/es/blog/star-schema-vs-snowflake-schema

En este procedimiento, crearemos tablas en el esquema `finance`, insertaremos datos en esas tablas y cubriremos los conceptos clave para aprovechar el **procesamiento masivamente paralelo** (**MPP**) y la arquitectura columnar.

Preparación

Para completar este procedimiento, necesitará todo lo mencionado en la sección *Requisitos técnicos* al principio del tema.

Cómo hacerlo...

Veamos cómo crear tablas en Amazon Redshift:

Creemos una tabla de clientes en el esquema `financiero` con los atributos `customer_number`, `first_name`, `last_name` y `date_of_birth`:

```
CREATE TABLE finance.customer
(
  customer_number   INTEGER,
  first_name        VARCHAR(50),
  last_name         VARCHAR(50),
  date_of_birth     DATE
);
```

Nota

Algo clave para crear una tabla de clientes es definir las columnas y sus tipos de datos correspondientes. Amazon Redshift admite tipos de datos como numéricos, caracteres, fecha, fecha y hora con zona horaria, booleanos, geometría, *HyperLog*, *log sketch*, super, etc.

Ahora insertaremos 10 registros en la tabla de clientes utilizando una instrucción `de inserción` de valores múltiples. El uso de instrucciones de inserción de varias filas es más eficiente que ejecutar instrucciones `de inserción` separadas para cada fila:

```
insert into finance.customer values
(1, 'foo', 'bar', '1980-01-01'),
(2, 'john', 'smith', '1990-12-01'),
 (3, 'spock', 'spock', '1970-12-01'),
 (4, 'scotty', 'scotty', '1975-02-01'),
 (5, 'seven', 'of nine', '1990-04-01'),
 (6, 'kathryn', 'janeway', '1995-07-01'),
 (7, 'tuvok', 'tuvok', '1960-06-10'),
 (8, 'john', 'smith', '1965-12-01'),
 (9, 'The Doctor', 'The Doctor', '1979-12-01'),
 (10, 'B Elana', 'Torres', '2000-08-01');
```

Ahora puede revisar la información sobre la tabla de clientes utilizando la vista del sistema `svv_table_info`. Ejecute la siguiente consulta:

```
select "schema", table_id, "table", encoded, diststyle, sortkey1, size, tbl_rows
from svv_Table_info
where "table" = 'customer'
and "schema" = 'finance';
```

Este es el resultado esperado:

```
schema  | table_id |  table   |     encoded     | diststyle |   sortkey1    | size | tbl_rows
---------+----------+----------+-----------------+-----------+---------------+------+----------
 finance |   108292 | customer | Y, AUTO(ENCODE) | AUTO(ALL) | AUTO(SORTKEY) |    7 |       10
```

`table_id` es el ID del objeto y el número de registros de la tabla es 10. La columna `encoded` indica que la tabla está comprimida. Amazon Redshift almacena las columnas en bloques inmutables de 1 **megabyte (MB)**. El tamaño de la tabla es de 7 MB. Cuando creó la tabla de `customer`, observe que no especificó ninguna clave. Cuando no se especifican claves, las tablas se crean con un estilo de distribución predeterminado de `AUTO` y una clave de ordenación de `AUTO`. `AUTO` indica que Amazon Redshift determinará de forma inteligente y automática las claves adecuadas para usted en función de su patrón de carga de trabajo y las actualizará a medida que esta evolucione. Profundicemos en la terminología y los conceptos de `diststyle` y `sortkey`:

- `diststyle` es una propiedad de la tabla que determina cómo se distribuyen los datos de esa tabla entre las secciones de datos de su almacén de datos de Amazon Redshift. Una sección de datos es una partición lógica en la que los datos se almacenan en caché en el SSD de Amazon Redshift.
- `KEY`: El valor se somete a un proceso de hash y el mismo valor va a la misma ubicación (segmento de datos) en el nodo de cálculo.
- `ALL`: los datos completos de la tabla se envían al primer segmento de cada nodo de cálculo.
- `EVEN`: utiliza un método *round-robin* para distribuir uniformemente las filas de la tabla entre todos los segmentos, lo que garantiza una distribución equilibrada de los datos independientemente del valor de la columna.

- `AUTO`: con la distribución `AUTO`, Amazon Redshift asigna un estilo de distribución óptimo en función del tamaño de los datos de la tabla. Por ejemplo, si se especifica el estilo de distribución `AUTO`, Amazon Redshift asigna inicialmente el estilo de distribución `ALL` a una tabla pequeña. Cuando la tabla crece, Amazon Redshift puede cambiar el estilo de distribución a `KEY`, eligiendo la clave principal (o una columna de la clave principal compuesta) como clave de distribución. Si la tabla crece y ninguna de las columnas es adecuada para ser la clave de distribución, Amazon Redshift cambia el estilo de distribución a `EVEN`. El cambio en el estilo de distribución se produce en segundo plano con un impacto mínimo en las consultas de los usuarios.

Ejecutemos una consulta en la tabla de clientes para enumerar los clientes nacidos antes de 1980:

```
select *
from finance.customer
where extract(year from date_of_birth) < 1980;
```

También puede crear una copia de la tabla permanente utilizando `create table as (CTAS)`. Ejecutemos la siguiente consulta para crear otra tabla para un cliente nacido en 1980:

```
create table finance.customer_dob_1980 as
select *
from finance.customer
where extract(year from date_of_birth) = 1980 ;
```

También puede crear tablas temporales, por ejemplo, para generar ID en una operación de carga de datos. Las tablas temporales solo se pueden consultar durante la sesión actual y se eliminan automáticamente cuando finaliza la sesión. Se crean en un esquema específico de la sesión y no son visibles para ningún otro usuario. Para ello, puede utilizar el comando `create temporary/temp table`. Ejecute las tres consultas siguientes en una sola sesión:

```
create temporary table #customer(custid integer IDENTITY(1,1), customer_number
integer;
insert into #customer (customer_number) values(1);
select * from #customer;
```

Este es el resultado esperado:

```
custid  customer_number
1 1
```

Vuelva a conectarse a Amazon Redshift utilizando el cliente SQL. Al volver a conectarse, se creará una nueva sesión. Ahora, intente ejecutar la siguiente consulta en la tabla temporal

```
#customer: select * from #customer;
```

Obtendrá el mensaje de error **`ERROR: 42P01: la relación «#customer» no existe`**, ya que las tablas temporales solo son visibles para la sesión actual.

Cómo funciona...

Cuando crea una tabla en Amazon Redshift, esta almacena los datos en el disco, columna por columna, en bloques de 1 MB. Amazon Redshift comprime las columnas de forma predeterminada, lo que reduce el espacio de almacenamiento y la **entrada/salida (E/S)** cuando ejecuta una consulta en la tabla. Amazon Redshift ofrece diferentes estilos de distribución para repartir los datos entre todos los nodos de cálculo, con el fin de aprovechar la arquitectura MPP

para su carga de trabajo. Los metadatos y la información resumida de la tabla se pueden consultar utilizando la tabla de catálogo y la vista resumida.

Amazon Redshift almacena metadatos sobre la tabla del cliente. Puede consultar la tabla de catálogo `pg_table_def` para recuperar esta información. Puede ejecutar la siguiente consulta para ver la estructura de la tabla/columna.

Nota importante

Cuando se insertan datos en una tabla, Amazon Redshift crea automáticamente, en la memoria, los metadatos de los valores `min` y `max` de cada bloque. Se accede a estos metadatos, conocidos como mapa de zona, antes de realizar un escaneo del disco para identificar qué bloques son relevantes para una consulta. Amazon Redshift no tiene índices, pero sí claves de ordenación. Las columnas de claves de ordenación controlan cómo se ordenan físicamente los datos de una tabla en el disco y pueden utilizarse como palanca para mejorar el rendimiento de las consultas.

Véase también...

Puede encontrar más información sobre los estilos de distribución en el siguiente enlace:

https://docs.aws.amazon.com/redshift/latest/dg/c_choosing_dist_sort.html.

4. Gestión de vistas en una base de datos

Los objetos de base de datos de vista permiten almacenar el resultado de una consulta. En Amazon Redshift, las vistas se ejecutan cada vez que se mencionan en una consulta. La ventaja de utilizar una vista en lugar de una tabla es que permite acceder solo a un subconjunto de datos de una tabla, unir más de una tabla en una sola tabla virtual y actuar como una tabla agregada, y no ocupa espacio en la base de datos, ya que solo se guarda la definición, lo que resulta muy práctico para abstraer consultas complicadas. En este procedimiento, crearemos vistas para almacenar consultas para las tablas subyacentes.

Cómo hacerlo...

Creemos una vista utilizando el comando `CREATE VIEW`. Seguiremos los siguientes pasos para crear una vista:

Crear una vista `finance.customer_vw` basada en los resultados de la consulta en `finance.customer`:

```
CREATE VIEW finance.customer_vw
AS
SELECT customer_number,
       first_name,
       last_name,
       EXTRACT(year FROM date_of_birth) AS year_of_birth
FROM finance.customer;
```

Puede crear vistas de enlace tardío utilizando la cláusula `WITH NO SCHEMA BINDING`. La cláusula especifica que la vista no está vinculada a los objetos de la base de datos subyacente, como tablas y funciones definidas por el usuario. Como resultado, no existe dependencia entre la vista y los objetos a los que hace referencia. Al no existir dependencia, puede eliminar o modificar un objeto al que se hace referencia sin afectar a la vista.

Amazon Redshift no comprueba las dependencias hasta que se consulta la vista:

```
CREATE VIEW finance.customer_lbvw
```

```
AS
SELECT customer_number,
       first_name,
       last_name,
       EXTRACT(year FROM date_of_birth) AS year_of_birth
FROM finance.customer
with no schema binding;
```

Para verificar que se ha creado una vista, puede utilizar el siguiente comando:

```
SELECT table_schema as schema_name,
       table_name as view_name,
       view_definition
FROM information_schema.views
WHERE table_schema not in ('information_schema', 'pg_catalog')
ORDER by schema_name,
         view_name;
```

Nota

Este *script* proporcionará una salida de las vistas creadas bajo un esquema particular y el *script* SQL para la vista.

Ahora podemos seleccionar directamente desde la vista `finance.customer_vw`, al igual que con cualquier otro objeto de la base de datos, de la siguiente manera:

```
SELECT * from finance.customer_vw limit 5;
```

Nota

Aquí, la vista `finance.customer_vw` abstrae la **información de identificación personal (PII)** `date_of_birth` de la tabla subyacente y proporciona al usuario una vista abstracta de solo los datos esenciales para ese año para determinar el grupo de edad.

Este es el resultado esperado:

```
outputcustomer_number,first_name,last_name,year_of_birth
1    foo    bar    1980
2    john    smith    1990
3    spock    spock    1970
4    scotty    scotty    1975
5    seven    of nine    1990
```

Para eliminar la vista creada anteriormente, puede utilizar el siguiente comando:

```
DROP VIEW finance.customer_vw ;
```

5. Gestión de vistas materializadas en una base de datos

Una vista materializada es un objeto de base de datos que persiste los resultados de una consulta en el disco. En Amazon Redshift, las vistas materializadas permiten almacenar consultas complejas de uso frecuente como objetos de base de datos independientes, lo que le permite acceder a estos objetos de base de datos directamente y obtener respuestas más rápidas a las consultas.

El uso de vistas materializadas es un enfoque común para potenciar las consultas repetibles en un panel de **inteligencia empresarial** (**BI**) y evita costosos cálculos cada vez. Además, las vistas materializadas permiten una actualización incremental de los resultados, utilizando los datos de la tabla subyacente. En este procedimiento, crearemos una vista materializada para consultar las tablas y también para conservar los resultados para una recuperación más rápida.

Cómo hacerlo...

Creemos una vista materializada utilizando el comando `CREATE MATERIALIZED VIEW`. Seguiremos los siguientes pasos para crear una vista materializada, con el fin de almacenar los resultados precalculados de una consulta analítica y también ver cómo actualizarla:

Cree una vista materializada `finance.customer_agg_mv` utilizando los resultados de la consulta basada en `finance.customer`. Puede optar por añadir una clave de distribución o claves de ordenación a las vistas materializadas, al igual que en las tablas. Si no lo especifica, el estilo de distribución predeterminado es `EVEN`.

Amazon Redshift puede actualizar la vista materializada automáticamente cuando se actualizan los datos en los objetos de datos subyacentes. Utilice la cláusula `AUTO REFRESH` para controlar este comportamiento. El valor predeterminado para `AUTO REFESH` es no:

```
CREATE MATERIALIZED VIEW finance.customer_agg_mv
AUTO REFRESH YES
AS
SELECT
       EXTRACT(year FROM date_of_birth) AS year_of_birth,
       count(1) customer_cnt
FROM finance.customer
group by EXTRACT(year FROM date_of_birth);
```

Ahora podemos seleccionar directamente desde `finance.customer`, al igual que con cualquier otro objeto de la base de datos, de la siguiente manera:

```
select * from finance.customer_agg_mv limit 5;
```

Este es el resultado esperado:

```
outputyear_of_birth,customer_cnt
1975    1
1979    1
1995    1
1970    1
1965    1
```

Puede verificar el estado de una vista materializada utilizando una tabla del sistema `SVV_MV_INFO` (**https://docs.aws.amazon.com/redshift/latest/dg/r_SVV_MV_INFO.html**):

```
select * from SVV_MV_INFO where name='customer_agg_mv';
```

Este es el resultado esperado:

```
Output:
database_name,schema,name,is_stale,owner_user_name,state,autorefresh,
autorewrite
vdwpoc       finance customer_agg_mv    f    vdwadmin 1    f    t
```

Aquí, `stale='f'` indica que los datos están actualizados, reflejando la tabla base subyacente `de costumer`. Esta columna se puede utilizar para actualizar la vista materializada cuando sea necesario. Otra columna clave en la tabla `SVV_MV_INFO` es la columna `state`, que indica si es posible una actualización incremental (`state=1`) o no (`state=0`). En la vista materializada, creamos un estado `state=1`, que indica que es posible una actualización incremental más rápida.

Ahora, carguemos más datos en los valores subyacentes `de finance.customer` utilizando el siguiente comando y comprobemos la tabla `STV_MV_INFO`:

```
insert into finance.customer values
(11, 'mark', 'bar', '1980-02-01'),
(12, 'pete', 'smith', '1990-2-01'),
 (13, 'woofy', 'spock', '1980-11-01'),
 (14, 'woofy jr', 'scotty', '1975-03-01'),
 (15, 'eleven', 'of nine', '1990-07-01');
```

Vuelva a consultar la vista `SVV_MV_INFO` para comprobar el estado de la vista materializada:

```
select name,is_stale,state from SVV_MV_INFO where name='customer_agg_mv';
```

Resultado:

```
name,is_stale,state
customer_agg_mv    t    1
```

Tenga en cuenta que `stale = 't'` indica que los datos subyacentes de la vista materializada han cambiado, pero es posible actualizarlos de forma incremental.

Actualice la vista materializada con el comando `REFRESH MATERIALIZED VIEW` y compruebe de nuevo el estado:

```
REFRESH MATERIALIZED VIEW finance.customer_agg_mv;
```

Este es el resultado esperado:

```
select name,is_stale, state from SVV_MV_INFO where name='customer_agg_mv';
```

Resultado:

```
name,is_stale,state
customer_agg_mv  f 1
```

Como podemos ver en el fragmento de código anterior, `customer_agg_mv` ahora se actualiza para reflejar los datos de la tabla subyacente.

Cómo funciona...

Una vista materializada se puede actualizar con los datos más recientes de las tablas subyacentes mediante el comando `REFRESH MATERIALIZED VIEW`. El propietario de la vista materializada puede invocar una actualización y también tener acceso `SELECT` a las referencias de las tablas en la vista materializada. Cuando se actualiza la vista materializada, se ejecuta una transacción independiente para actualizar el conjunto de datos. Amazon Redshift también admite una opción de actualización automática para mantener la vista materializada actualizada lo antes posible después de que cambien las tablas base.

Amazon Redshift ofrece la función **Automated Materialized Views** (**AutoMV**) para mejorar el rendimiento de las consultas mediante la creación y gestión automáticas de vistas materializadas

basadas en la supervisión de la carga de trabajo y algoritmos de aprendizaje automático. Estas son algunas de las características clave de AutoMV:

- **Supervisión continua**: Amazon Redshift supervisa continuamente la carga de trabajo mediante técnicas de aprendizaje automático para identificar oportunidades de mejora del rendimiento mediante la creación de vistas materializadas.
- **Creación y eliminación automáticas**: cuando el sistema detecta que una vista materializada sería beneficiosa, la crea y la mantiene automáticamente. Por el contrario, si una AutoMV creada anteriormente ya no proporciona beneficios de rendimiento, el sistema la elimina automáticamente.
- **Sin impacto en la carga de trabajo del usuario**: la función AutoMV solo se activa durante los periodos de baja actividad de los usuarios en el clúster. Si inicia una carga de trabajo mientras se está ejecutando una operación AutoMV, la tarea AutoMV se detendrá para liberar recursos para la carga de trabajo del usuario. Esto garantiza que sus cargas de trabajo tengan prioridad sobre las operaciones AutoMV.

Sus usuarios no necesitan saber nada de AutoMV. Pueden seguir usando las tablas base y Amazon Redshift reescribirá automáticamente esas consultas para usar AutoMV y mejorar el rendimiento de las consultas. Los desarrolladores no necesitan revisar las consultas para aprovechar AutoMV.

6. Gestión de procedimientos almacenados en una base de datos

Los procedimientos almacenados en Amazon Redshift son objetos creados por el usuario utilizando el lenguaje de programación procedural **Procedural Language/PostgreSQL (PL/pgSQL)**. Los procedimientos almacenados admiten tanto **el lenguaje de definición de datos (DDL)** como **el lenguaje de manipulación de datos (DML)**. Los procedimientos almacenados pueden aceptar argumentos de entrada, pero no necesariamente tienen que devolver resultados. **PL/pgSQL** también admite lógica condicional, bucles y sentencias case. Los procedimientos almacenados se utilizan habitualmente para crear canalizaciones de datos reutilizables **de extracción, transformación y carga (ETL)** y para ayudar al **administrador de bases de datos (DBA)** a automatizar actividades administrativas rutinarias, como eliminar periódicamente las tablas que no se utilizan.

El atributo `SECURITY` controla quién tiene privilegios para acceder a determinados objetos de la base de datos.

Los procedimientos almacenados se pueden crear con controles de definición de seguridad para permitir la ejecución de un procedimiento sin dar acceso a las tablas subyacentes; por ejemplo, pueden eliminar una tabla creada por otro usuario y ayudar al DBA a automatizar las actividades administrativas. Este procedimiento trata sobre la gestión de procedimientos almacenados en una base de datos.

El atributo `NONATOMIC` controla la atomicidad del procedimiento almacenado. Si no se especifica `NONATOMIC`, el valor predeterminado es atómico, lo que significa que todas las sentencias del procedimiento almacenado deben tener éxito o se revertirá todo el procedimiento. En el modo no atómico, cada sentencia SQL dentro del procedimiento almacenado se ejecuta y se confirma de forma independiente. Si una sentencia falla, las sentencias anteriores permanecen confirmadas.

Cómo hacerlo...

En este procedimiento, comenzaremos creando una UDF escalar basada en Python que se utilizará para analizar una entrada **de lenguaje de marcado extensible (XML)**:

Conéctese a Amazon Redshift utilizando el cliente SQL y copie y pegue el siguiente código para crear un procedimiento almacenado `sp_cookbook`:

Crear esquema:

```
Create schema cookbook;
```

Crear procedimiento:

```
create or replace procedure sp_cookbook(indate in date, records_out INOUT
refcursor) as
$$
declare
  integer_var int;
begin
   RAISE INFO 'running first cookbook storedprocedure on date %',  indate;
   drop table if exists cookbook.cookbook_tbl;
  create table cookbook.cookbook_tbl
  (recipe_name varchar(50),
   recipe_date date
   );
   insert into cookbook.cookbook_tbl values('stored procedure', indate);
  GET DIAGNOSTICS integer_var := ROW_COUNT;
   RAISE INFO 'rows inserted into cookbook_tbl = %', integer_var;
    OPEN records_out FOR SELECT * FROM cookbook.cookbook_tbl;
END;
$$ LANGUAGE plpgsql;
```

Este procedimiento almacenado toma dos **parámetros**: `indate` es la entrada y `records_out` sirve tanto como parámetro de entrada como de salida. Este procedimiento almacenado utiliza sentencias DDL y DML. El usuario actual es el propietario del procedimiento almacenado y también es el propietario de la tabla `cookbook.cookbook_tbl`.

Descripción del código del procedimiento:

Definición del procedimiento

`create or replace procedure sp_cookbook(indate in date, records_out INOUT refcursor) as`

- create or replace procedure → Crea el procedimiento o lo reemplaza si ya existe.
- sp_cookbook → Nombre del procedimiento.
- indate in date → Parámetro de entrada (IN) de tipo date.
- records_out INOUT refcursor →
  - Parámetro de entrada/salida (INOUT).
  - Tipo refcursor → devolverá un cursor con un result set.
- as → Indica que empieza el cuerpo del procedimiento.

$$

Delimitador del bloque del procedimiento.
Todo lo que esté entre $$ y $$ es el cuerpo en PL/pgSQL.

Sección DECLARE

```
declare
  integer_var int;
```

- Se declara una variable local llamada integer_var
- Tipo int
- Se usará para guardar el número de filas afectadas

Inicio del bloque ejecutable

```
begin
```

Comienza la lógica del procedimiento.

Mensaje informativo

```
RAISE INFO 'running first cookbook storedprocedure on date %',  indate;
```

- RAISE INFO → Muestra un mensaje en el log/consola.
- % → Placeholder para insertar el valor de indate.

Ejemplo de salida:

```
INFO: running first cookbook storedprocedure on date 2026-03-01
```

Elimina tabla si existe

```
drop table if exists cookbook.cookbook_tbl;
```

- Borra la tabla cookbook_tbl si ya existe.
- Evita error si no existe.
- Está en el esquema cookbook.

Crea tabla

```
create table cookbook.cookbook_tbl
(
  recipe_name varchar(50),
  recipe_date date
);
```

Crea una tabla con:

- recipe_name → texto hasta 50 caracteres
- recipe_date → tipo fecha

Inserta datos

```
insert into cookbook.cookbook_tbl
values('stored procedure', indate);
```

Inserta:

- Texto fijo: 'stored procedure'
- La fecha recibida como parámetro (indate)

Obtiene número de filas afectadas

```
GET DIAGNOSTICS integer_var := ROW_COUNT;
```

- ROW_COUNT guarda el número de filas afectadas por el último INSERT.
- Se almacena en integer_var.

En este caso será 1.

Muestra filas insertadas

```
RAISE INFO 'rows inserted into cookbook_tbl = %', integer_var;
```

Imprime algo como:

```
INFO: rows inserted into cookbook_tbl = 1
```

Abre el cursor de salida

```
OPEN records_out FOR
SELECT * FROM cookbook.cookbook_tbl;
```

- Abre el cursor records_out.
- Ejecuta el SELECT.
- Devuelve el resultado al llamador.

Esto permite hacer:

```
CALL sp_cookbook('2026-03-01', 'mycursor');
FETCH ALL FROM mycursor;
```

Fin del bloque

```
END;
```

Termina el bloque BEGIN.

```
$$ LANGUAGE plpgsql;
```

- Cierra el delimitador $$.
- Indica que el lenguaje es plpgsql.

En resumen:

- Recibe una fecha.
- Borra y crea una tabla.
- Inserta una fila con esa fecha.
- Muestra mensajes informativos.
- Devuelve el contenido de la tabla mediante un cursor.

Nota

Algunas versiones antiguas de las herramientas de cliente SQL pueden dar error con cadenas entre comillas y sin terminar en o cerca de «$$». Asegúrese de que tiene la última versión del cliente SQL.

Ahora, ejecutemos un procedimiento almacenado `sp_cookbook` utilizando la instrucción `call` y recuperemos la salida del cursor resultante utilizando la instrucción `fetch`. Resalte ambas instrucciones y ejecútelas:

```
call sp_cookbook(current_date, 'inputcursor');
fetch all from inputcursor;
```

Advertencia:

Las sentencias `fetch all` están deshabilitadas en un clúste monodo como el nuestro.

Este sería el resultado esperado:

```
Message
running first cookbook storedprocedure on date 2020-12-13
rows inserted into cookbook_tbl = 1
recipe_name    recipe_date
```

```
stored procedure      2020-12-13 00:00:00
```

Para ver una definición del procedimiento almacenado creado anteriormente, puede ejecutar la siguiente instrucción:

```
SHOW PROCEDURE sp_cookbook(indate in date, records_out INOUT refcursor);
```

Ahora crearemos otro procedimiento almacenado con un privilegio de definidor de seguridad:

```
create or replace procedure public.sp_self_service(tblName in varchar(60)) as
$$
begin
    RAISE INFO 'running sp_self_service to drop table %',  tblName;
    execute 'drop table if exists cookbook.' || tblName;
    RAISE INFO 'table dropped %',  tblName;
END;
$$ LANGUAGE plpgsql
SECURITY DEFINER;
```

Descripción del código del procedimiento

Definición

`create or replace procedure public.sp_self_service(tblName in varchar(60)) as`

- create or replace procedure → crea o reemplaza el procedimiento.
- public.sp_self_service → nombre del procedimiento en el esquema public.
- tblName in varchar(60) → parámetro de entrada:
    - Nombre de la tabla a eliminar.
    - Hasta 60 caracteres.

`$$`

Delimitador del cuerpo del procedimiento.

Inicio del bloque

`begin`

Comienza la lógica ejecutable.

Mensaje informativo

`RAISE INFO 'running sp_self_service to drop table %',  tblName;`

- Imprime en logs el nombre de la tabla que se intentará eliminar.
- % se reemplaza por el valor de tblName.

Ejemplo:

INFO: running sp_self_service to drop table ventas_2024

SQL dinámico

`execute 'drop table if exists cookbook.' || tblName;`

Aquí está lo importante.

- execute → ejecuta SQL dinámico.
- Se construye una cadena:

```
'drop table if exists cookbook.' || tblName
```

- Si tblName = 'ventas_2024', ejecuta:

```
drop table if exists cookbook.ventas_2024;
```

👉 Esto permite que el nombre de la tabla sea variable.

Segundo mensaje

```
RAISE INFO 'table dropped %',  tblName;
```

Confirma en logs que la tabla fue eliminada (o intentada).

Fin del bloque

```
END;
```

Termina la lógica.

```
$$ LANGUAGE plpgsql
SECURITY DEFINER;
```

LANGUAGE plpgsql

Indica que usa PL/pgSQL.

🔐 **SECURITY DEFINER**

Este punto es clave:

Significa que el procedimiento se ejecuta con los **permisos del usuario que lo creó**, no del usuario que lo llama.

👉 Esto permite que:

- Un usuario sin permisos para DROP TABLE
- Pueda ejecutar el procedimiento
- Y la tabla se elimine usando los privilegios del creador

En resumen:

1. Recibe un nombre de tabla.
2. Ejecuta un DROP TABLE dinámico en el esquema `cookbook`.
3. Se ejecuta con privilegios elevados (si el creador los tiene).
4. Registra mensajes informativos.

Creemos un usuario y comprobemos si tiene permiso para eliminar la tabla `cookbook.cookbook_tbl`. El usuario `user1` no tiene permiso para eliminar la tabla:

```
create user user1 with password 'Cookbook1';


-- Otorga a user1 permiso para ejecutar el procedimiento sp_self_service.
grant execute on procedure public.sp_self_service(tblName in varchar(60)) to
user1;
```

```
-- Cambia el usuario activo de la sesión a user1.
-- A partir de aquí, todo se ejecuta como si fueras user1.
set SESSION authorization user1;
select current_user;

-- Si user1 no tiene permisos sobre esa tabla,
-- Esta sentencia fallará con error de permisos.
drop table cookbook.cookbook_tbl;
```

Este es el resultado esperado:

```
ERROR: 42501: permission denied for schema cookbook
```

Cuando `user1` ejecuta el procedimiento almacenado `sp_self_service`, el procedimiento se ejecuta con la seguridad del propietario del procedimiento:

```
set SESSION authorization  user1;
select current_user;
call public.sp_self_service('cookbook_tbl');
```

Este es el resultado esperado:

```
running sp_self_service to drop table cookbook_tbl
table
```

Esto permite al usuario eliminar la tabla sin proporcionar todos los permisos para las tablas del esquema `cookbook`.

Cómo funciona...

Amazon Redshift utiliza el lenguaje procedimental PL/pgSQL para crear los procedimientos almacenados. PL/pgSQL proporciona acceso programático que se puede utilizar para crear estructuras de control para el lenguaje SQL y permitir cálculos complejos. Por ejemplo, si tiene un procedimiento almacenado que puede crear usuarios y configurar el acceso necesario que satisfaga las necesidades de su organización, en lugar de invocar varios comandos, ahora esto se puede hacer en un solo paso. El atributo de acceso `SECURITY` de un procedimiento almacenado define los privilegios para acceder a los objetos de la base de datos subyacente utilizados. De forma predeterminada, se utiliza un `INVOKER` que utiliza los privilegios del usuario, y el `SECURITY DEFINER` permite al usuario del procedimiento utilizar los privilegios del propietario.

Véase también...

Puede encontrar la referencia completa al lenguaje procedimental PL/pgSQL en **https://www.postgresql.org/docs/8.0/plpgsql.html#PLPGSQL-ADVANTAGES**. Los procedimientos almacenados útiles y listos para usar se pueden encontrar en **https://github.com/awslabs/amazon-redshift-utils/tree/master/src/StoredProcedures**.